Personalized Tourism Management System

Comprehensive Technical Reference Manual, Software Architecture, Algorithmic Logic & File-by-File Blueprint

Stack: Java 17 LTS | Swing + FlatLaf | MySQL 8.0 | JDBC | OpenPDF | JFreeChart | jBCrypt

Architecture: Layered MVC / DAO / Service Architecture

Team Composition: 7 Engineers (Dev 1 through Dev 8 roles)

Active Repository: 48 Java Classes, 6 Database Tables, 11 Curated Destinations

1. Executive Summary & Architectural Overview

The **Personalized Tourism Management System** is a production-grade enterprise desktop application engineered using **Java 17 (LTS)** and modern **Swing (FlatLaf)**. It delivers an intelligent travel planning platform featuring rule-based package recommendations, automated multi-day itinerary generation, dynamic multi-currency conversions, step-by-step booking with dual-mode payment gateways (Credit Card & UPI), cancellation with automated refund estimation, cryptographic user authentication (jBCrypt), digital boarding pass generation, downloadable OpenPDF itineraries/invoices, and an administrative portal with visual analytics powered by JFreeChart.

Core Architectural Layers & Design Patterns

- **Strict 3-Tier Layered Architecture:** Strict separation of concerns: Presentation Layer (ui.*) → Business Logic Layer (service.*) → Data Access Layer (dao.*) → Database (MySQL). UI classes are completely forbidden from executing raw SQL queries.
- **Singleton Pattern:** Implemented in DBConnection for thread-safe centralized database connection pooling, UserSession for tracking logged-in credentials across the desktop lifecycle, and CurrencyManager / LanguageManager for global locale synchronization.
- **Strategy Pattern:** Utilized in the payment subsystem where the PaymentMethod interface defines processPayment(), implemented concretely by CardPayment (Luhn-checked card validation, masking) and UpiPayment (Virtual Payment Address validation).
- **Observer / Listener Pattern:** Employed in CurrencyManager where UI panels register listeners to dynamically re-render package prices, striking previous prices and updating symbols upon currency switching without application restart.
- **Transfer Object / POJO Pattern:** Clean entity representation across all layers (User, Destination, Package, Booking, ItineraryDay, Review, CustomerBookingDossier).

Request Pipeline:
[Swing UI: Event Dispatch Thread (EDT)] → [Service Validation & Rules Engine] → [DAO PreparedStatements ('?' Bindings)] → [MySQL Database Engine]
[Data Display]: [MySQL ResultSets] → [Model POJOs] → [Service Aggregation/PDF/Charts] → [FlatLaf Custom Renderers]

2. Team Work Breakdown & Developer Contributions

The project was executed by a collaborative engineering team with well-defined module ownership, formal Git feature branches, and Pull Request (PR) peer reviews merged into the develop branch. The table below documents what each developer delivered:

Developer & Role	Git Identity & Commits	Modules Owned & Specific Files Contributed	Key Responsibilities & Algorithmic Deliverables
Amy / Amy Ann Jims **Team Lead & Full-Stack Architect**	amyjims2103@gmail.com 25 commits PR #1, #3-#6 Merges	• pom.xml, tourism_schema.sql • DBConnection.java, ThemeConstants.java • CurrencyManager.java, LanguageManager.java • UserDAO.java, AdminDAO.java, AuthService.java • LoginFrame.java, RegisterFrame.java • CustomerMainFrame.java, AdminMainFrame.java • BoardingPassDialog.java, CustomerBookingsAdminPanel.java • CustomerDossierDialog.java, Main.java	Foundational project scaffolding; database DDL schema and seed data for 11 global destinations; BCrypt user authentication and registration; central Swing UI wiring connecting all 8 developer modules; warm terracotta visual identity system; real-time dynamic multi-currency converter; digital boarding pass with custom ticket geometry; administrative customer roster & masked settlements inspector; headless JVM fallback and terminal diagnostics; code review and pull request integration.
Aein Shaji **Dev 3: Catalog & Browsing**	aeinshaji16@gmail.com Commit 3a05858 Branch: feature/catalog	• DestinationDAO.java • PackageDAO.java (catalog queries) • CatalogPanel.java • PackageCard.java (catalog layout) • CatalogMockPreview.java	Constructed travel package browsing catalog; implemented multi-attribute filtering (category chips, keyword search, price range); responsive card grid layout displaying destination hero photos, duration badges, discount indicators, and 'View Itinerary' / 'Book Now' trigger actions; mock preview harness for visual card testing.
Gowri P **Dev 4: Recommendation & Itinerary**	gowripthampi@gmail.com Commit 3a0ef1e PR #1 Merged	• RecommendationService.java • ItineraryDAO.java • ItineraryService.java • RecommendationsPanel.java • ItineraryDialog.java • Dev4VerificationTest.java	Designed rule-based recommendation mathematical engine scoring packages (0-100%) across budget, duration, and category interest; built interactive preferences questionnaire with sliders and check chips; built day-by-day itinerary schedule retriever and timeline dialog view; wrote unit verification test suite.
Garry George **Dev 5: Booking & Payments**	garry.george.5212@gmail.com Commit 1527a7f PR #3 Merged	• BookingDAO.java • BookingService.java • BookingException.java • PaymentMethod.java • CardPayment.java, UpiPayment.java • BookingDialog.java • BookingHistoryPanel.java	Engineered complete booking lifecycle; dynamic price calculator factoring guests and discounts; strict date validation preventing past bookings; payment strategy implementation for Card and UPI; 48-hour cancellation policy with automated 80% refund computation; customer booking history dashboard with status pill tags and cancellation actions.
Pius Varghese **Dev 6: Admin Inventory CRUD**	piusvarghese12@gmail.com Commit b719bc8 PR #6 Merged	• InventoryManagementPanel.java • BookingOversightPanel.java	Built full CRUD administrative panel for travel packages and destinations using JTable with modal Add/Edit/Delete dialogs and input validation; constructed platform-wide booking oversight monitoring table allowing administrators to inspect all customer transactions and status overrides.
Ashwin V **Dev 7: PDF Reports & Analytics**	kunjachuv@gmail.com Commit 95db05d PR #4 Merged	• PDFExportService.java • AnalyticsService.java • ReportsPanel.java	Integrated OpenPDF (LGPL) to generate branded, printable customer itineraries and official tax invoices; authored SQL aggregation queries for destination popularity and net monthly revenues; built visual administrative analytics dashboard using JFreeChart (Bar Chart and Revenue Breakdown).

Atul **Dev 8: Reviews & Notifications**	atul.moniv@gmail.com Commit 7a8b7ba PR #5 Merged	• ReviewDAO.java • ReviewDialog.java • Notification banner in CustomerMainFrame	Implemented customer package review and rating engine (1-5 stars) linked to confirmed trips; created interactive star-rating modal dialog; built in-app upcoming trip reminder banner alerting customers upon login if an active departure is within 7 days; visual rating star harmonizer.

3. Core Business Logic, Algorithms & Mathematical Models

The application incorporates several specialized business logic algorithms across recommendation matching, cost estimation, cancellation refund guarantees, and cryptographic authentication:

A. Rule-Based Recommendation Scoring Algorithm (RecommendationService)

Rather than relying on black-box heuristics, packages are scored transparently against user criteria using a weighted normalized multi-attribute utility model:

Scoring Formula:

FinalScore = (0.40 × BudgetMatch + 0.35 × DurationMatch + 0.25 × InterestMatch) × 100

1. Budget Match Component (w = 0.40):

• Estimated Trip Cost = (Package Final Price × Guests) + (Estimated Daily Cost × Duration × Guests)

• If Estimated Cost ≤ Budget: BudgetMatch = 1.0 - (SavingsRatio × 0.25) (Rewards affordability while utilizing budget).

• If Estimated Cost > Budget: BudgetMatch = max(0.0, 1.0 - (Overspend / MaxTolerance)) where MaxTolerance = Budget × 0.30. A penalty is applied proportionally up to a 30% overspend cutoff, yielding 0 beyond that.

2. Duration Match Component (w = 0.35):

• DiffDays = |PackageDuration - DesiredDays|

• DurationMatch = max(0.0, 1.0 - (DiffDays / DesiredDays))

3. Category / Interest Match Component (w = 0.25):

• If user selects no preference: InterestMatch = 0.70 (Neutral).

• If category matches user selection: InterestMatch = 1.00.

• If category does not match: InterestMatch = 0.15.

B. Pricing Calculation & 48-Hour Cancellation Refund Policy (BookingService)

1. Dynamic Price Calculation:

Total = (BasePrice × Guests) × (1.0 - DiscountPercentage / 100.0)

Guarantees exact 2-decimal rounded calculation for multi-guest groups.

2. Cancellation Refund Guarantee:

MinutesRemaining = Duration.between(Now, TravelDate.atStartOfDay()).toMinutes()

• If MinutesRemaining > 48 hours × 60 min: **Refund Amount = TotalCost × 80%**

• If MinutesRemaining ≤ 48 hours × 60 min: **Refund Amount = Rs. 0.00 (0% refund)**

• Trips already in the past (TravelDate < Now) cannot be cancelled under any circumstance.

C. Cryptographic Password Hashing (AuthService & jBCrypt)

Passwords are never stored in plaintext. During registration, BCrypt.hashpw(plainTextPassword, BCrypt.gensalt(12)) generates a salted one-way hash with a work factor of 12 (4,096 iterations). During login, BCrypt.checkpw(enteredPassword, storedHash) verifies credentials securely resisting rainbow table and timing attacks.

4. Database Architecture & Relational Schema (MySQL)

The relational backend consists of 6 normalized tables hosted on MySQL 8.0, with foreign key integrity, cascade deletion constraints, and unique indexes:

Table Name	Primary Key	Key Columns & Types	Foreign Keys & Integrity Constraints
users	user_id (INT AUTO)	full_name VARCHAR(100), email VARCHAR(100) UNIQUE, password_hash VARCHAR(255), role ENUM('CUSTOMER', 'ADMIN'), phone VARCHAR(20)	Unique index on email prevents duplicate accounts.
destinations	destination_id (INT AUTO)	name VARCHAR(100), country VARCHAR(100), description TEXT, category ENUM('Adventure', 'Beach', 'Culture', 'Nature', 'Luxury'), estimated_daily_cost DECIMAL(10,2), image_url VARCHAR(255)	Categorizes regions for recommendation filtering; maps local images in resources.
travel_packages	package_id (INT AUTO)	destination_id INT, title VARCHAR(150), duration_days INT, base_price DECIMAL(10,2), season VARCHAR(50), discount_percentage DECIMAL(5,2), is_active BOOLEAN	FOREIGN KEY (destination_id) REFERENCES destinations(destination_id) ON DELETE CASCADE
itinerary_days	itinerary_id (INT AUTO)	package_id INT, day_number INT, title VARCHAR(150), activity_description TEXT	FOREIGN KEY (package_id) REFERENCES travel_packages(package_id) ON DELETE CASCADE
bookings	booking_id (INT AUTO)	user_id INT, package_id INT, travel_date DATE, number_of_guests INT, total_cost DECIMAL(10,2), booking_status ENUM('CONFIRMED', 'CANCELLED', 'COMPLETED'), payment_status VARCHAR(50), refund_amount DECIMAL(10,2)	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE FOREIGN KEY (package_id) REFERENCES travel_packages(package_id) ON DELETE CASCADE
reviews	review_id (INT AUTO)	user_id INT, package_id INT, rating INT (1-5), comment TEXT, review_date TIMESTAMP	FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE CHECK (rating BETWEEN 1 AND 5)

5. Comprehensive File-by-File Catalog (What Each File Does)

Below is the complete inventory of all 48 source files, configuration classes, resources, and tests comprising the project. Each entry details the file's architectural role, core methods, dependencies, and design patterns:

File Path & Class	Category	Architecture Role & Detailed Functionality
DBConnection.java com/tourism/config	**Config / Database**	Thread-safe Singleton JDBC connection manager. Reads MySQL connection string (`jdbc:mysql://localhost:3306/tourism_db`), user (`tourism_user`), and password (`tourism123`). Contains reconnect logic and prevents connection leak by reusing the active connection instance.
ThemeConstants.java com/tourism/config	**Config / UI Theme**	Centralized design system defining visual palette: Warm Sunset Coral (`#E05638`), Deep Terracotta (`#B2361C`), Emerald Green (`#27AE60`), Crimson (`#C0392B`), Charcoal Text (`#272B33`), Segoe UI font hierarchy (large header, subheaders, body, small), and padding standards.
CurrencyManager.java com/tourism/config	**Config / Locale**	Thread-safe Singleton currency converter supporting USD, EUR, GBP, INR, JPY, AUD, CAD with dynamic exchange rates and symbol formatting. Implements the Observer pattern allowing UI widgets to subscribe to currency toggle events without restarting the application.
LanguageManager.java com/tourism/config	**Config / i18n**	Localization manager supporting English, Spanish, French, German, Hindi, and Japanese translations for UI navigation labels, status banners, and system actions.

User.java com/tourism/model	**Model / Entity**	POJO encapsulating customer and administrator identity: `userId`, `fullName`, `email`, `passwordHash`, `role` (`CUSTOMER` vs `ADMIN`), `phone`, and `createdAt` timestamp.
Destination.java com/tourism/model	**Model / Entity**	Entity representing geographic travel locations: `destinationId`, `name`, `country`, `description`, `category` (`Adventure`, `Beach`, `Culture`, `Nature`, `Luxury`), `estimatedDailyCost`, and image asset path.
Package.java com/tourism/model	**Model / Entity**	Core travel package model: `packageId`, `destinationId`, `title`, `durationDays`, `basePrice`, `discountPercentage`, and `isActive`. Provides `getFinalPrice()` computing net discounted rate and optional references to hotel/transportation.
ItineraryDay.java com/tourism/model	**Model / Entity**	Encapsulates individual day schedule for a package: `itineraryId`, `packageId`, `dayNumber`, `title`, and `activityDescription` detailing daily excursions.
Booking.java com/tourism/model	**Model / Entity**	Represents customer reservation: `bookingId`, `userId`, `packageId`, `travelDate`, `numberOfGuests`, `totalCost`, `bookingStatus` (`CONFIRMED`, `CANCELLED`, `COMPLETED`), `paymentStatus`, and `refundAmount`.
Review.java com/tourism/model	**Model / Entity**	Encapsulates customer feedback: `reviewId`, `userId`, `packageId`, numeric `rating` (1-5 stars), `comment` text, and `reviewDate`.
Hotel.java com/tourism/model	**Model / Entity**	Represents lodging details: `hotelId`, `name`, `starRating`, `pricePerNight`, `address`, and amenity list.
Transportation.java com/tourism/model	**Model / Entity**	Encapsulates transit options: `transportId`, `type` (Flight, Train, Luxury Bus), `provider`, `departureLocation`, `arrivalLocation`, and fare.
CustomerBookingDossier.java com/tourism/model	**Model / Admin Composite**	Composite administrative entity joining customer registration details, package selection, travel date, guests, masked payment tokens, and fulfillment state for the Admin Dossier Inspector.
UserSession.java com/tourism/model	**Model / Session**	Thread-safe Singleton holding the currently authenticated `User` object, tracking active login lifecycle and role permissions across the desktop session.
UserDAO.java com/tourism/dao	**DAO / Persistence**	Executes parameterized SQL queries for user authentication and management: `findByEmail()`, `registerUser()`, `findById()`, `emailExists()`, and `updateProfile()`.
DestinationDAO.java com/tourism/dao	**DAO / Persistence**	CRUD operations for destinations: `findAll()`, `findById()`, `findByCategory()`, `insert()`, `update()`, and `delete()`.
PackageDAO.java com/tourism/dao	**DAO / Persistence**	Package data layer: `findAll()`, `findAllActive()` (eagerly joining destination data), `findById()`, `searchPackages()`, and administrative CRUD operations (`createPackage`, `updatePackage`, `deletePackage`).
ItineraryDAO.java com/tourism/dao	**DAO / Persistence**	Itinerary persistence: `findByPackageId()` retrieving sorted day-by-day activities, `insert()`, and cascade deletion helpers.
BookingDAO.java com/tourism/dao	**DAO / Persistence**	Booking database queries: `createBooking()`, `getUserBookings(userId)`, `getAllBookings()`, `cancelBooking(bookingId, refund)`, and `markPastTripsCompleted()` automated status updater.
ReviewDAO.java com/tourism/dao	**DAO / Persistence**	Customer review persistence: `createReview()`, `findByPackageId()`, `getAverageRating()`, and `hasUserReviewedPackage()` preventing duplicate feedback.
AdminDAO.java com/tourism/dao	**DAO / Persistence**	Specialized administrative audit queries joining customer rosters with booking fulfillment status, travel dates, and payment settlement updates (`updatePaymentStatus`, `updateBookingStatus`).
AuthService.java com/tourism/service	**Service / Security**	Authentication business logic: performs input validation (email regex, password length), verifies passwords using `BCrypt.checkpw()`, and registers users with salted `BCrypt.hashpw()`.
RecommendationService.java com/tourism/service	**Service / Algorithm**	Core rule-based recommendation engine. Implements the multi-factor scoring formula across budget, duration, and category interest, sorting packages in descending order with human-readable rationale.

ItineraryService.java com/tourism/service	**Service / Workflow**	Manages itinerary retrieval, day sequencing, and dynamic synthesis of placeholder daily activities if customized schedules are not stored in database.
BookingService.java com/tourism/service	**Service / Business Logic**	Booking orchestration: validates future dates and guest limits, calculates final discounted price, invokes payment strategy validation, creates booking records, and enforces the 48-hour 80% refund cancellation rule.
BookingException.java com/tourism/service	**Service / Exception**	Domain-specific checked exception for invalid bookings, overcapacity, past dates, or payment failures.
PaymentMethod.java com/tourism/service	**Service / Strategy**	Strategy design pattern interface declaring `validate()` and `processPayment(double amount, String reference)`.
CardPayment.java com/tourism/service	**Service / Strategy**	Credit/Debit card payment strategy: validates 16-digit card number, MM/YY expiry, CVV, and produces masked receipt tokens (`•••• •••• •••• 1234`).
UpiPayment.java com/tourism/service	**Service / Strategy**	UPI payment strategy: validates Virtual Payment Address format (`user@bank`) via regular expression and generates unique transaction sequence IDs.
PDFExportService.java com/tourism/service	**Service / PDF Engine**	OpenPDF document generator: constructs formatted, printable customer travel itineraries with day breakdown tables and official booking invoices with billing breakdown and customer details.
AnalyticsService.java com/tourism/service	**Service / Analytics**	Administrative business intelligence service: aggregates non-cancelled bookings to compute top destinations and calculates net monthly revenue (`total_cost - refund_amount`) for JFreeChart rendering.
Main.java com/tourism	**UI / Entry Point**	Application bootstrap class. Sets FlatLaf Look and Feel, verifies MySQL database connectivity, detects graphical display environment (`DISPLAY` / `WAYLAND_DISPLAY`), launches `LoginFrame` on the EDT, or prints helpful headless CLI diagnostics.
LoginFrame.java com/tourism/ui/auth	**UI / Authentication**	Split-pane modern login window: left brand showcase with travel badges; right form with email/password fields, role detection, remember me, error alerts, and links to registration.
RegisterFrame.java com/tourism/ui/auth	**UI / Authentication**	Customer registration modal with full name, email, phone, password, and confirmation validation; automatically hashes passwords and redirects to login.
CustomerMainFrame.java com/tourism/ui/customer	**UI / Customer Portal**	Main customer dashboard hosting sidebar navigation tabs (Catalog, Recommendations, My Bookings), currency chooser, language chooser, light/dark theme toggle, user profile card, and 7-day upcoming trip reminder banner.
CatalogPanel.java com/tourism/ui/customer	**UI / Customer Portal**	Interactive catalog browser featuring search bar, category quick-filter chips (Beach, Adventure, Culture, Nature, Luxury), budget filters, and a responsive grid layout of package cards.
PackageCard.java com/tourism/ui/customer	**UI / Customer Portal**	Individual card widget displaying destination hero photo, title, duration pill badge, category chip, live weather badge, star rating, strike-through base price, discounted price, and action buttons.
RecommendationsPanel.java com/tourism/ui/customer	**UI / Customer Portal**	Personalized questionnaire UI with sliders for budget, duration, guest count, and category check chips; triggers `RecommendationService` and renders ranked recommendation cards with match percentage tags.
ItineraryDialog.java com/tourism/ui/customer	**UI / Customer Portal**	Modal dialog presenting day-wise timeline with destination photos, activity descriptions, and 'Download PDF Itinerary' button invoking `PDFExportService`.
BookingDialog.java com/tourism/ui/customer	**UI / Customer Portal**	Modal booking dialog with date picker, guest spinner, live price recalculator, payment method tabs (Credit Card vs UPI), input validation, and booking confirmation confirmation.
BookingHistoryPanel.java com/tourism/ui/customer	**UI / Customer Portal**	Customer reservation dashboard showing past and upcoming trips, status pill badges (`CONFIRMED`, `CANCELLED`, `COMPLETED`), cancellation button with real-time refund calculation, 'Download Invoice', and 'Boarding Pass' trigger.

BoardingPassDialog.java com/tourism/ui/customer	**UI / Customer Portal**	Digital boarding pass modal styled like an airline flight ticket with route codes, passenger name, seat/gate info, perforation cutouts, and rendered barcode.
ReviewDialog.java com/tourism/ui/customer	**UI / Customer Portal**	Interactive feedback dialog featuring 1-5 star rating selector, comment text area, and submission validation linked to completed trips.
AdminMainFrame.java com/tourism/ui/admin	**UI / Admin Portal**	Administrative console navigation frame hosting Inventory Management, Booking Oversight, Customer Dossier & Settlements, and Analytics & Reports tabs with theme toggle and logout.
InventoryManagementPanel.java com/tourism/ui/admin	**UI / Admin Portal**	Administrative package CRUD interface: searchable `JTable` of packages and destinations with Add, Edit, and Delete action dialogs and form validation.
BookingOversightPanel.java com/tourism/ui/admin	**UI / Admin Portal**	Master booking oversight table displaying all platform transactions, guest counts, total revenue, status filters, and administrative cancellation override.
CustomerBookingsAdminPanel.java com/tourism/ui/admin	**UI / Admin Portal**	Enterprise customer roster displaying registered travelers, selected travel packages, masked payment settlements, and trigger button for Dossier Inspector.
CustomerDossierDialog.java com/tourism/ui/admin	**UI / Admin Portal**	Detailed modal inspector displaying comprehensive traveler identity, booked package metadata, itinerary timeline, and masked payment tokens.
ReportsPanel.java com/tourism/ui/admin	**UI / Admin Portal**	Visual business intelligence dashboard incorporating JFreeChart: Top Destinations Bar Chart, Revenue Breakdown Pie Chart, and KPI metric summary cards.
tourism_schema.sql src/main/resources	**Database / DDL**	MySQL database initialization script creating database, 6 tables with foreign keys, default administrative account (`admin@tourism.com`), sample customer (`john@gmail.com`), 11 global destinations, and day-by-day itineraries.
***.jpg** images	**Assets / Media**	Curated high-resolution destination photographs for Bali, Swiss Alps, Kyoto, Paris, Kullu-Manali, Goa, Shimla, Kerala, Vietnam, Maldives, and Egypt.
VerifyAuth.java com/tourism	**Test / Verification**	Automated unit test verifying BCrypt hash generation, password verification, and UserDAO authentication logic.
Dev3VerificationTest.java com/tourism	**Test / Verification**	Unit test suite validating package catalog queries and active package filtering.
Dev4VerificationTest.java com/tourism	**Test / Verification**	Algorithmic verification test asserting correct recommendation scoring, duration matching, and budget overspend penalties.
CatalogMockPreview.java com/tourism/ui/customer	**Test / Harness**	Standalone Swing test harness for visual inspection of `PackageCard` layout, weather badges, and price formatting.

6. End-to-End System Workflows

Customer Journey (From Discovery to Boarding Pass)

- **1. Registration / Authentication:** User signs up in RegisterFrame with BCrypt-hashed password or logs in via LoginFrame; authenticated session is cached in UserSession.
- **2. Catalog Discovery & Recommendations:** User explores packages in CatalogPanel using category filter chips and search, or opens RecommendationsPanel to submit budget, duration, and style preferences to receive scored matches.
- **3. Day-by-Day Itinerary Inspection:** User clicks 'View Itinerary' on any PackageCard to inspect daily activities in ItineraryDialog and optionally clicks 'Download PDF Itinerary' to save an OpenPDF document.
- **4. Booking & Payment Processing:** User clicks 'Book Now', selects travel date and guest count; enters Credit Card or UPI credentials in BookingDialog; price is computed and verified before writing to MySQL.

- **5. Reservation Management & Boarding Pass:** User views bookings in BookingHistoryPanel, clicks 'Boarding Pass' to render digital pass with barcode in BoardingPassDialog, downloads official tax invoice PDF, or cancels with 80% refund if > 48 hours.

Administrator Journey (Inventory Management & Analytics)

- **1. Secure Admin Access:** Admin signs in with verified ADMIN role, navigating directly to AdminMainFrame.
- **2. Inventory Control (CRUD):** In InventoryManagementPanel, admin creates, edits, discounts, or disables travel packages with real-time MySQL synchronization.
- **3. Booking Oversight & Status Overrides:** In BookingOversightPanel, admin reviews every platform booking and overrides reservation states when required.
- **4. Customer Dossier & Payment Settlements:** In CustomerBookingsAdminPanel, admin views customer roster, inspecting individual dossiers in CustomerDossierDialog with masked payment hashes.
- **5. Business Intelligence & Reporting:** In ReportsPanel, admin analyzes JFreeChart Top Destinations Bar Chart, monthly revenue trends, and overall booking volume KPIs.

7. Build, Verification & Execution Reference

1. Prerequisites: JDK 17 LTS, Apache Maven 3.8+, MySQL Server 8.0+ running on port 3306.

2. Database Setup:

```
CREATE USER IF NOT EXISTS 'tourism_user'@'localhost' IDENTIFIED BY 'tourism123';
CREATE DATABASE IF NOT EXISTS tourism_db;
GRANT ALL PRIVILEGES ON tourism_db.* TO 'tourism_user'@'localhost'; FLUSH PRIVILEGES;
mysql -u tourism_user -ptourism123 tourism_db < src/main/resources/tourism_schema.sql
```

3. Maven Build & Desktop Execution:

```
mvn clean compile
mvn exec:java
```

4. Headless Terminal Fallback:
If executed over SSH or without a display server, Main.java gracefully detects lack of graphical environment, runs database diagnostics, and prints active destination/package counts without throwing unhandled exceptions.

Document Prepared For: Academic & Technical Review | **Architecture Status:** Fully Integrated & Tested (Java 17 / FlatLaf / MySQL 8.0)